Concurrency: the largest unmodelled term, fitted and validated

Generated by `scripts/analysis/concurrency_model.py`. Raw numbers: `results/concurrency_model_metrics.json`, per-query CV: `results/concurrency_model_loqo.csv`.

Every other prediction in this project is single-client. Production is not. This fits and honestly cross-validates a concurrency term on 22 TPC-H queries at 1, 2, 4 and 8 simultaneous clients (Apple M1, 8 cores).

The model

`T(N) = T(1) · N^e`, with `e` fitted per query by least squares on log N against log T. `e = 0` means the machine absorbs the extra load for free; `e = 1` means fully serialised.

median elasticity	**0.734**
range	**0.286 to 0.984**
power-law fit quality	median R^2 **0.944**, min 0.701
poorly fitting queries ($R^2 < 0.9$)	7 of 22 — q2, q4, q10, q11, q14, q19, q22

The power law is a reasonable but imperfect description: a third of the queries deviate from it noticeably, so it should be presented as a one-parameter approximation, not a law.

Can elasticity be predicted without running under load?

No. Leave-one-query-out, predicting `e` from features available from a single-client run (cost, rows, buffer hits/reads, I/O ratio, single-client runtime):

method	MAE(e)	RMSE(e)
global constant (median of training queries)	**0.1672**	**0.2143**
plan features (random forest)	0.1910	0.2351

Plan features are worse than a constant. Concurrency elasticity is not recoverable from a single-client plan, so a predictor cannot infer it and the load factor has to be supplied by the caller. That is the honest product behaviour and it is what `cetp` now does.

What a global constant is nonetheless worth

Mean absolute percentage error on runtime under load, predicting `T(N)` from `T(1)` and an estimate of `e`:

method	N=2	N=4	N=8
ignore concurrency entirely	33.90%	56.09%	**73.79%**
global constant e = 0.734	24.34%	35.73%	**41.81%**
plan-predicted e	20.44%	30.29%	42.64%
oracle (the query's own fitted e)	26.29%	25.38%	**6.72%**

Three things follow.

1. A global constant roughly halves the error at 8 clients (73.79% → 41.81%). Worth applying by default, which is why `DEFAULT_CONCURRENCY_ELASTICITY = 0.734` exists.

2. 41.81% is still large — bigger than the whole cross-environment error the rest of this project works on. Concurrency is not "handled" by a default; it is only reduced.

3. The oracle reaches 6.72% at 8 clients. The gap between 41.81% and 6.72% is the price of not knowing *this* query's elasticity. So elasticity is strongly query-specific and highly informative — it simply cannot be guessed from a plan. A query whose behaviour under load actually matters should be measured under load, once; that is a far cheaper experiment than anything else in this project.

How it is wired into the product

`cetp.operator_model.predict_scaling(..., prod_clients=N, concurrency_elasticity=e)` applies `N^e`. Behaviour is deliberately loud:

- `prod_clients=1` (the default) emits **SINGLE-CLIENT** as a warning, because a silent single-client assumption is how this term got ignored for the whole project.
- Supplying `prod_clients` without an elasticity uses the default and says so, quoting the 41.8%-against-6.7% gap, so nobody mistakes a population median for a measurement of their query.

Caveats

- **One machine.** Elasticity depends on core count and the query mix as much as on the query, so 0.734 is the median for *this* 8-core machine on *this* workload. It does not transfer to a machine with a different core count without re-measurement, and the product should not pretend otherwise.
- 4 repetitions per (query, client) with the first discarded; the reported latency is the median across clients, i.e. what one client experiences, which is what an SLA is written against. Throughput-oriented questions need a different statistic.
- All clients run the *same* query. A realistic mixed workload is untested and would plausibly behave differently, since contention depends on what else is running.